

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.18

Missing Values & Duplicates

Day 19 — every choice has a cost

90 min

Concepts + live code

30 min

Practice notebook

0

Free choices

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 18

1 What is merge's default how?

→ inner — a silent filter.

2 Can a merge ADD rows?

→ Yes. Duplicate keys multiply them.

3 What does indicator=True give?

→ both / left_only / right_only.

4 Why did my int column become float?

→ An unmatched row put a NaN in it.

5 `concat(axis=1)` matches on what?

→ The index — not a key.

Yesterday's merges CREATED missing values and duplicate rows. Today is about what to do with them.

How today's two hours are structured

1

Finding what is missing

And the gaps `isna()` cannot see

20 min

2

Dropping

`dropna`, and when it is honest

15 min

3

Filling

`fillna`, `ffill`, `interpolate` — and the cost

25 min

4

Duplicates

Exact, partial, and fuzzy

20 min

5

Mini build

A cleaning pipeline that reports itself

5 min

6

Practice notebook

You code, I walk around

30 min

Section 1 is red because the worst missing values are the ones that do not look missing. Section 3 because filling always distorts something.

By the end of today you will be able to

1

Find

Locate missing values, including the ones disguised as data

2

Decide

Choose between dropping and filling, and justify it

3

Measure

Say what your cleaning did to the mean, the spread and the row count

4

Deduplicate

Remove exact and near-duplicate rows safely

There is no neutral option

Dropping rows loses information and can bias what remains. Filling invents numbers that were never measured. Leaving gaps makes every later step harder. All three are legitimate; none is free. The professional skill is not picking the right one — it is stating which you picked, why, and what it cost.

1

Finding What Is Missing

Including what `isna()` cannot see · 20 minutes

- The three commands
- Disguised missing values
- `na_values`

Three commands, run on every new dataset

Command	Gives	Why it matters
<code>df.isna().sum()</code>	missing count per column	the headline number
<code>df.isna().mean()</code>	the PROPORTION per column	5 missing of 10 is very different from 5 of 10,000
<code>df.isna().any(axis=1).sum()</code>	rows with any gap	how many rows a dropna() would remove

Read the proportion, not just the count

```
a    0.20    one in five  – fillable
b    0.60    most of it   – probably drop the COLUMN
c    0.00    clean
```

A column that is 60% empty rarely deserves to be filled. Filling it would mean inventing more data than you measured, and every later model would be learning your assumption rather than the world.

Is it missing at random?

Before deciding what to do, ask WHY the values are missing. If high earners skip the salary question, dropping those rows removes the high earners and your average falls — the gap is not random, so neither is the damage.

```
# compare the rows WITH and WITHOUT the gap
df[df["salary"].isna()][ "dept"].value_counts()
df[df["salary"].notna()][ "dept"].value_counts()
```

The gaps `isna()` cannot see

⚠ These are all “missing” — and none are NaN

	name	age	score	city
0	Ravi	34	88	Pune
1	Sara	-999	91	N/A
2	Amit	29	NaN	Delhi
3	Neha	41	unknown	Pune

```
df.isna().sum() -> age 0    score 1    city 1
# it found ONE of the four. -999, N/A and unknown are just data.
```

The fix, at read time

```
pd.read_csv(path, na_values=[
    "-999", "N/A", "unknown",
    "", "-", "missing"])
```

Now `isna()` finds all four — and a bonus: the score column becomes float64 instead of text, because the word “unknown” was what forced it to be a string in the first place.

How to spot them before they bite

```
df["city"].value_counts()    look for odd entries
df["age"].describe()        a min of -999 is a flag
df.dtypes                   text where you expected numbers
```

A numeric column stored as text almost always means a sentinel value is hiding in it. A minimum of -999, or a maximum of 9999, is a placeholder somebody chose decades ago. Run `value_counts` and `describe` on every column before you trust any of them.



Dropping

When removing rows is honest · 15 minutes

- dropna
- how, thresh, subset
- The cost

dropna — and the arguments that make it precise

Call	Removes	Use it when
<code>df.dropna()</code>	any row with ANY gap	rarely — it is the bluntest option
<code>df.dropna(how="all")</code>	only completely empty rows	always safe; a good first step
<code>df.dropna(subset=["salary"])</code>	rows missing THAT column	the usual choice
<code>df.dropna(thresh=3)</code>	rows with fewer than 3 real values	when rows need a minimum of evidence
<code>df.dropna(axis=1)</code>	whole COLUMNS with any gap	when a column is mostly empty

⚠ dropna() is greedier than it looks

```
5 rows, one gap in column a, three in column b
df.dropna()      -> 2 rows survive
```

A single gap anywhere removes the whole row, including the columns that were perfectly fine.

When dropping IS the right answer

- Very few rows affected, and they look random
- The missing column is the thing you are measuring
- You will report how many you removed

3

Filling

Every fill distorts something · 25 minutes

- fillna
- ffill & interpolate
- Measuring the cost

Five ways to fill a gap

Call	Puts in	Right for
<code>fillna(0)</code>	zero	counts, where missing truly means none
<code>fillna(s.mean())</code>	the average	roughly symmetric numeric data
<code>fillna(s.median())</code>	the middle value	skewed data, or data with outliers
<code>s.fffll()</code>	the previous value	ordered data: sensor readings, prices
<code>s.interpolate()</code>	a value on the line between	smooth numeric series over time

⚠ `fillna(0)` is almost never right for a measurement

```
[10, 20, NaN, 40, 50]    true mean 30.0
fillna(0)                mean 24.0
```

A missing temperature is not 0 degrees. A missing salary is not an unpaid employee. Zero is a VALUE, and inserting it changes the answer.

⚠ `fffll` assumes an ORDER

Carrying the previous value forward only makes sense if the rows are in a meaningful sequence — by date, by time, by reading number. On a table sorted alphabetically by name, `fffll` copies one stranger's data onto another. Sort first, or do not use it.

Measure what your filling did

The same five numbers, filled three ways

```
s = [10, 20, NaN, 40, 50]
```

Choice	mean	std	n	What it did
leave it	30.0	18.26	4	honest, but 4 of 5 values
<code>fillna(mean)</code>	30.0	15.81	5	mean held, SPREAD SHRANK
<code>fillna(0)</code>	24.0	19.49	5	the mean is now wrong
<code>interpolate()</code>	30.0	15.81	5	guessed 30 — here, correctly

⚠ Mean-filling shrinks the spread

Every filled value sits exactly at the centre, so it adds no variation at all. The standard deviation fell from 18.26 to 15.81 — the data now looks MORE consistent than it really is, and any model trained on it will be overconfident.

The three lines to print every time

```
before: mean, std, count  
after : mean, std, count  
filled: how many, with what
```

If you cannot state those numbers, you do not know what you did.



Duplicates

Exact, partial, and fuzzy · 20 minutes

- duplicated
- subset & keep
- Near-duplicates
- Order matters

Finding and removing duplicate rows

	id	name	v
0	1	A	10
1	2	B	20
2	2	B	20
3	3	C	30
4	3	C-diff	31

```
df.duplicated()      [F F True F F]   only row 2
df.drop_duplicates() 4 rows – row 2 removed
# row 4 is NOT an exact duplicate: name and v differ
```

subset changes the question

```
drop_duplicates()      4 rows
drop_duplicates(subset="id") 3 rows
```

Without subset, a row counts as a duplicate only if EVERY column matches. With subset="id", id 3 appears twice, so one of them goes — and you have just silently chosen which version of C to keep.

keep= decides WHICH copy survives

```
keep="first"  the default – keeps the earliest
keep="last"   keeps the most recent
keep=False    removes ALL copies, keeping none
```

keep="last" is often what you want for records updated over time. keep=False is for INVESTIGATING — use `df[df.duplicated(subset="id", keep=False)]` to look at every conflicting copy before deciding.

Near-duplicates, and why order matters

! These four are two cities

```
["Pune", " Pune", "PUNE", "Mumbai"]  
  
nunique() -> 4  
str.strip().str.lower()  
    .nunique() -> 2
```

drop_duplicates compares strings exactly, so a leading space makes two identical cities look different. Day 15's staff.csv had exactly this.

! And remember the Day 15 trap

```
str.strip().str.title()  
"IT" -> "It" mangled  
str.strip().str.lower()  
"IT" -> "it" safe
```

Normalising to lower case is predictable and never invents capitalisation. Cleaning can introduce errors as easily as it removes them — always re-inspect afterwards.

The order of your cleaning steps changes the answer

```
["Pune", " Pune", "Delhi"]  
  
dedupe, then clean -> 3 rows  
clean, then dedupe -> 2 rows
```

Deduplicating first leaves “Pune” and “ Pune” as separate rows, because at that moment they genuinely are different strings. Normalise the text FIRST, then deduplicate. This is a real ordering rule, not a style preference — the two pipelines give different data.

Debug corner — eight cleaning bugs

You wrote	What happens	Fix
<code>df.isna().sum()</code> only	misses -999, "N/A", "unknown"	<code>read_csv(na_values=[...])</code>
<code>df.dropna()</code>	one gap removes the whole row	<code>dropna(subset=["col"])</code>
<code>df.fillna(0)</code> on a measurement	the mean becomes wrong	<code>fillna(median)</code>
<code>fillna(mean)</code> unreported	the spread silently shrinks	print std before and after
<code>ffill()</code> on unsorted rows	copies a stranger's value	sort first, or do not use it
<code>df.drop_duplicates()</code>	misses " Pune" vs "Pune"	strip and lower FIRST
dedupe, then clean text	leaves near-duplicates behind	clean, THEN dedupe
<code>df.dropna()</code> without assigning	nothing changes — it returns a copy	<code>df = df.dropna(...)</code>

Row 8 catches everyone at least once: almost every cleaning method RETURNS a new frame rather than changing yours.

Putting it together — a pipeline that reports itself

```
import pandas as pd

# ---- 1. read, declaring the disguises up front
df = pd.read_csv("staff.csv",
                 na_values=["-999", "N/A", "unknown", ""])
start = len(df)
print("rows:", start)
print(df.isna().mean().round(2))      # proportion per column

# ---- 2. normalise text BEFORE deduplicating
for col in ["name", "dept"]:
    df[col] = df[col].str.strip().str.lower()

# ---- 3. now duplicates are findable
dups = df.duplicated(subset=["emp_id"], keep=False)
print("conflicting rows:\n", df[dups])      # LOOK before dropping
df = df.drop_duplicates(subset=["emp_id"], keep="last")

# ---- 4. fill, and measure what it cost
before = df["salary"].agg(["mean", "std", "count"])
df["salary"] = df["salary"].fillna(df["salary"].median())
after = df["salary"].agg(["mean", "std", "count"])
print(pd.concat([before, after], axis=1,
                keys=["before", "after"]).round(2))

# ---- 5. the receipt
print(f"{start} rows in, {len(df)} out")
```



na_values first

catch the disguised gaps at read time



Proportion, not count

5 of 10 differs from 5 of 10,000



Normalise, THEN dedupe

order changes the answer



keep=False to LOOK

inspect conflicts before dropping



median, not mean

safer when outliers exist



before/after table

proves what the fill cost



Row receipt

every pipeline ends with the count

Anyone reading this output can reconstruct exactly what you did to the data. That is the standard to aim for.

Practice session — 30 minutes



Predict the output

Disguised gaps, dropna, fill costs, dedupe

8 questions



Spot & fix the bug

Zero-filling, unsorted ffill, wrong order

4 questions



Write the code

Find, drop, fill, measure, deduplicate

5 tasks

Your notebook

[Day19_Practice_Questions.ipynb](#)

- value_counts before trusting
- Print mean and std before/after
- Raise a hand — I am walking around
- Solutions shared after class

Every cleaning step today must be followed by one line saying what it changed: how many rows, and what happened to the mean and spread.

TODAY IN TWELVE LINES

Recap

- 1 There is no neutral choice — dropping and filling both cost.
- 2 `isna()` misses -999, "N/A" and "unknown". Use `na_values`.
- 3 A numeric column stored as text means a sentinel is hiding.
- 4 Read the PROPORTION missing, not just the count.
- 5 Ask WHY it is missing before deciding what to do.
- 6 `dropna()` removes a row for a gap in any column at all.
- 7 `dropna(subset=[...])` is almost always the better call.
- 8 `fillna(0)` on a measurement changes the mean. Avoid it.
- 9 `fillna(mean)` holds the mean but SHRINKS the spread.
- 10 `ffill` assumes an order — sort first, or do not use it.
- 11 `drop_duplicates: subset=` decides what counts, `keep=` which survives.
- 12 Normalise text BEFORE deduplicating. The order changes the answer.

Homework

Take a messy CSV and write a cleaning pipeline that prints a receipt · Fill one column three different ways and compare the mean and std each time · Find a dataset where dropping the gaps biases the result, and say how

Next class — Topic 1.19

Outlier detection & treatment: the 1.5 x IQR rule, z-scores, and deciding whether a strange value is an error or the finding